

Algoritmos e Programação

Capítulo 3 — Comandos de Condição

Fonte: MEDINA, Marco. Algoritmos e Programação: Teoria e Prática.

Até aqui, todos os algoritmos tinham fluxo único: as instruções eram sempre executadas na mesma ordem. Os comandos de condição (ou comandos de decisão) alteram o fluxo de execução do algoritmo, permitindo que diferentes instruções sejam executadas de acordo com a avaliação de um valor booleano.

■ Analogia do livro — Ir ao mercado: se tenho mais dinheiro, compro produtos mais caros; caso contrário, compro marcas mais baratas. O dinheiro determina a ação e a forma da compra.

Este capítulo apresenta os dois principais comandos de decisão: se e caso.

3.1 — Comando se

Sintaxe geral

```
se <valor booleano> então
    <bloco de instruções 1>
[senão
    <bloco de instruções 2>]
<continuação do algoritmo>
```

- Colchetes [] indicam que a cláusula **senão** é **opcional**.
- <valor booleano> pode ser uma variável booleana, uma constante (V ou F), ou qualquer expressão cujo resultado seja lógico.

Semântica

Valor da condição	Fluxo executado
V (verdadeiro)	Executa bloco 1, pula bloco 2, segue para continuação
F (falso)	Pula bloco 1, executa bloco 2 (se existir), segue em frente

■ Operadores relacionais produzem booleanos — Expressões como $a = 5$, $idade \geq 18$, $n1 > n2$ retornam V ou F — são exatamente o tipo esperado pelo se.

3.1.1 — Verifica5 (sem senão)

Portugol

```

Algoritmo Verifica5_Versão1

var a: inteiro;
início
1■ escreva("Entre com um valor: ");
2■ leia(a);
3■ se (a = 5) então          { a = 5 retorna um valor lógico }
    início
4■         escreva("Condição Verdadeira");
5■         escreva("O valor é ", a);
    fim
6■ escreva("Fim do algoritmo.");
fim

```

Teste de mesa — a = 5 (condição V)

Instrução	Linha	a
1	1	?
2	2	[5]
3	3	5
4	4	5
5	5	{5}
6	6	5

Teste de mesa — a = 2 (condição F)

Instrução	Linha	a
1	1	?
2	2	[2]
3	3	2
4	6	2

■ Leitura da tabela — Quando a condição é falsa, o algoritmo salta da linha 3 direto para a linha 6. As linhas 4 e 5 não são executadas. A coluna Instrução numera a ordem cronológica da execução.

Implementação em C

```

#include <stdio.h>

int main(void) {
    int a;
    printf("Entre com um valor: ");
    scanf("%d", &a);
    if (a == 5) {
        printf("Condição Verdadeira\n");
        printf("O valor é %d\n", a);
    }
    printf("Fim do algoritmo.\n");
    return 0;
}

```

■ Pegadinha clássica em C — = atribui; == compara. if (a = 5) atribui 5 a a e avalia a expressão como verdadeira (5 ≠ 0). Compiladores modernos avisam com -Wall.

3.1.2 — Verifica5 (com senão)

Portugol

```
Algoritmo Verifica5_Versão2

var a: inteiro;
início
1■ escreva("Entre com um valor: ");
2■ leia(a);
3■ se (a = 5) então
    início
4■         escreva("Condição Verdadeira");
5■         escreva("O valor é ", a);
        fim
    senão
        início
6■         escreva("Condição Falsa");
7■         escreva("O valor não é 5, é ", a);
        fim
8■ escreva("Fim do algoritmo.");
fim
```

Teste de mesa — a = 5 (bloco 1)

Instrução	Linha	a
1	1	?
2	2	[5]
3	3	5
4	4	5
5	5	{5}
6	8	5

Teste de mesa — a = 2 (bloco 2)

Instrução	Linha	a
1	1	?
2	2	[2]
3	3	2
4	6	2
5	7	{2}
6	8	2

■ Conclusões — Quando V, o bloco do senão não é executado. Quando F, o bloco do se não é executado. Em nenhuma execução ambos os blocos rodam.

3.1.3 — Problema 5: Maior de idade

Enunciado: Leia a idade de uma pessoa e diga se é maior de idade. Represente o valor que indica maioridade como uma constante valendo 18.

Portugol

```
Algoritmo Maior_Idade

constante LIMITE = 18;
var idade: inteiro;
    maior: booleano;
início
1■ escreva("Entre com a idade: ");
2■ leia(idade);
3■ se (idade >= LIMITE) então
4■     maior ← V;
    senão
5■     maior ← F;
6■ se (maior) então
7■     escreva("É maior de idade.");
    senão
8■     escreva("É menor de idade.");
fim
```

■ Três observações importantes — 1) se (maior) então equivale a se (maior = V) então. 2) Blocos com apenas um comando não precisam de início/fim. 3) A endentação é o que delimita visualmente o bloco subordinado ao se.

Implementação em C

```
#include <stdio.h>
#include <stdbool.h>

#define LIMITE 18

int main(void) {
    int idade;
    bool maior;
    printf("Entre com a idade: ");
    scanf("%d", &idade);
    if (idade >= LIMITE)
        maior = true;
    else
        maior = false;
    if (maior)
        printf("É maior de idade.\n");
    else
        printf("É menor de idade.\n");
    return 0;
}
```

■ Simplificação idiomática em C — Todo o primeiro if/else pode ser reduzido a: `bool maior = (idade >= LIMITE);` — o operador `>=` já produz um bool em C99+.

3.1.4 — Problema 6: Maior entre dois números

Enunciado: Leia dois inteiros e determine qual é maior. Considere que serão diferentes.

Portugol

```
Algoritmo Maior_Versão1

var n1, n2: inteiro;
início
1■ escreva("Entre com o primeiro valor: ");
2■ leia(n1);
3■ escreva("Entre com o segundo valor: ");
4■ leia(n2);
5■ se (n1 > n2) então
6■     escreva("O valor maior é: ", n1);
    senão
7■     escreva("O valor maior é: ", n2);
fim
```

3.1.3 — Condições Compostas

Até agora as condições eram únicas. Em problemas mais complexos, são necessárias condições combinadas pelos operadores lógicos e, ou, não (vistos no Cap. 2).

Problema 7 — Versão 1 (ifs independentes)

Enunciado: Leia três números e determine quantos são iguais.

Portugol

```
Algoritmo Iguais_Diferentes_Versão1

var n1, n2, n3: inteiro;
início
1■ escreva("Entre com o primeiro valor: ");
2■ leia(n1);
3■ escreva("Entre com o segundo valor: ");
4■ leia(n2);
5■ escreva("Entre com o terceiro valor: ");
6■ leia(n3);
7■ se (n1 = n2 e n2 = n3) então
8■     escreva("Os três valores são iguais");
9■ se (n1 <> n2 e n1 <> n3 e n2 <> n3) então
10■     escreva("Não há valores iguais");
11■ se (n1=n2 e n1<>n3) ou (n1=n3 e n1<>n2) ou (n2=n3 e n1<>n2) então
12■     escreva("Há dois valores iguais e um diferente");
13■ escreva("Fim do programa");
fim
```

■ Erro comum de iniciante — A comparação `se (n1 = n2 = n3)` é sintaticamente errada. Comparações entre três elementos devem usar conectivos lógicos: `se (n1 = n2 e n2 = n3) então`. Como a igualdade é transitiva, um dos termos é redundante.

Teste de mesa — $n1 = n2 = n3 = 2$

Instrução	Linha	n1	n2	n3
1	1	?	?	?
2	2	[2]	?	?
3	3	2	?	?

Instrução	Linha	n1	n2	n3
4	4	2	[2]	?
5	5	2	2	?
6	6	2	2	[2]
7	7	2	2	2
8	8	2	2	2
9	9	2	2	2
10	11	2	2	2
11	13	2	2	2

■ Problema da Versão 1 — A condição da linha 7 foi verdadeira, mas as condições das linhas 9 e 11 ainda foram testadas desnecessariamente. Total: 11 instruções.

Mapa de operadores: Portugol → C

Portugol	C	Significado
=	==	Igualdade
<>	!=	Diferente
e	&&	E lógico (AND)
ou		OU lógico (OR)
não	!	NÃO lógico (NOT)

3.1.4 — Comandos de Condição Encadeados

Um comando de decisão é um bloco de instruções. Se dentro dele houver outros comandos de decisão, temos encadeamento (aninhamento).

Caso particular: senão se

```

início
    ...
    se <valor booleano> então
        <bloco 1>
    senão se <valor booleano> então
        <bloco 2>
    senão se <valor booleano> então
        <bloco 3>
    senão
        ...
    <continuação do algoritmo>
fim

```

■ Quando usar encadeamento — Útil quando sabemos que apenas uma de várias condições será verdadeira — as demais sequer precisam ser testadas.

Problema 7 — Versão 2 (encadeada + condições compostas)

Portugol

```
Algoritmo Iguais_Diferentes_Versão2

var n1, n2, n3: inteiro;
início
1■ escreva("Entre com o primeiro valor: ");
2■ leia(n1);
3■ escreva("Entre com o segundo valor: ");
4■ leia(n2);
5■ escreva("Entre com o terceiro valor: ");
6■ leia(n3);
7■ se (n1 = n2 e n2 = n3) então
8■     escreva("Os três valores são iguais");
senão se (n1 <> n2 e n1 <> n3 e n2 <> n3) então
9■     escreva("Não há valores iguais");
senão
10■     escreva("Há dois valores iguais e um diferente");
11■ escreva("Fim do programa.");
fim
```

■ Por que sumiu a última condição composta? — A condição "dois iguais e um diferente" não precisa ser escrita: se as duas primeiras condições falharam, os números necessariamente são dois iguais e um diferente (as outras possibilidades já foram eliminadas).

Teste de mesa — $n1 = n2 = n3 = 2$

Instrução	Linha	n1	n2	n3
1	1	?	?	?
2	2	[2]	?	?
3	3	2	?	?
4	4	2	[2]	?
5	5	2	2	?
6	6	2	2	[2]
7	7	2	2	2
8	8	2	2	2
9	12	2	2	2

■ Comparação quantitativa — Versão 1: 11 instruções executadas. Versão 2: 9 instruções. A cláusula senão não tem custo computacional (não é uma instrução executada).

Problema 7 — Versão 3 (encadeada sem condições compostas)

Portugol

```

Algoritmo Iguais_Diferentes_Versão3

var n1, n2, n3: inteiro;
início
1■ escreva("Entre com o primeiro valor: ");
2■ leia(n1);
3■ escreva("Entre com o segundo valor: ");
4■ leia(n2);
5■ escreva("Entre com o terceiro valor: ");
6■ leia(n3);
7■ se (n1 = n2) então
8■     se (n2 = n3) então
9■         escreva("Os três valores são iguais");
        senão
10■         escreva("Há dois valores iguais e um diferente");
        senão { n1 <> n2 }
11■     se (n1 = n3) então
12■         escreva("Há dois valores iguais e um diferente");
        senão se (n2 = n3) então { n1 <> n3 }
13■         escreva("Há dois valores iguais e um diferente");
        senão { n2 <> n3 }
14■         escreva("Todos os valores são diferentes");
15■ escreva("Fim do programa.");
fim

```

■ Observações — Não há condições compostas nesta versão; tudo é resolvido com se/senão aninhados. Os comentários { n1 <> n2 }, { n1 <> n3 }, { n2 <> n3 } documentam o que é garantido pelo fluxo em cada ponto (invariantes).

3.1.4.1 — Problema 8: Maior entre dois (tratando iguais)

Enunciado: Leia dois inteiros e determine qual é maior. Se forem iguais, avise.

Portugol

```

Algoritmo Maior_Versão2

var n1, n2: inteiro;
início
1■ escreva("Entre com o primeiro valor: ");
2■ leia(n1);
3■ escreva("Entre com o segundo valor: ");
4■ leia(n2);
5■ se (n1 = n2) então
6■     escreva("Os valores devem ser diferentes!");
    senão se (n1 > n2) então
7■         escreva("O valor maior é: ", n1);
    senão
8■         escreva("O valor maior é: ", n2);
fim

```

3.2 — Comando caso

O comando caso é uma especialização do se encadeado, criado para simplificar um caso particular: quando comparamos uma mesma variável (inteiro ou caractere) contra vários valores constantes.

Sintaxe

```
caso <valor inteiro/caractere> seja:
    <primeiro valor>:
        <bloco de instruções>
    <segundo valor>:
        <bloco de instruções>
    ...
default:
    <bloco de instruções final>
```

Equivalência com se encadeado

```
se (<valor testado> = <primeiro valor>) então
    <bloco de instruções>
senão se (<valor testado> = <segundo valor>) então
    <bloco de instruções>
senão se ....
senão
    <bloco de instruções final>
```

■ Restrição do caso — Só funciona com valores constantes de tipo inteiro ou caractere. Não aceita reais nem expressões relacionais (ex.: $n1 > 0$ não cabe em um caso). Para essas situações, use se encadeado.

3.2.1 — Problema 9: Identificar se é 1, 5 ou 10

Portugol

```
Algoritmo Exemplo_Caso

var n1: inteiro;
início
    escreva("Entre com o valor a ser testado: ");
    leia(n1);
    caso n1 seja:
        1:
            escreva("Valor introduzido é válido: 1");
        5:
            escreva("Valor introduzido é válido: 5");
        10:
            escreva("Valor introduzido é válido: 10");
        default:
            escreva("Valor inválido: ", n1);
fim
```

Implementação em C

```
#include <stdio.h>

int main(void) {
    int n1;
    printf("Entre com o valor a ser testado: ");
    scanf("%d", &n1);
    switch (n1) {
        case 1: printf("Valor válido: 1\n"); break;
        case 5: printf("Valor válido: 5\n"); break;
        case 10: printf("Valor válido: 10\n"); break;
        default: printf("Valor inválido: %d\n", n1);
    }
    return 0;
}
```

■ **break obrigatório em C** — Sem break, o C executa o case e continua executando os seguintes (fall-through). Isso não acontece no caso do português, que sempre sai após o bloco.

■ **Uso típico: menus** — O caso é ideal para construir menus em que cada opção aciona uma função do programa.

3.2.2 — Problema 10: Menu de identificação

Enunciado: Faça um algoritmo que identifique se um número é: 1) par ou ímpar; 2) positivo, negativo ou zero; 3) ano bissexto.

Regras do ano bissexto

- Se o ano for divisível por **400** → bissexto.
- Senão, deve ser divisível por **4** e **não** ser divisível por **100** → bissexto.
- Caso contrário → não bissexto.

Condição composta: bissexto ■ $(\div 4 \wedge \neg \div 100) \vee (\div 400)$

Portugol

Algoritmo Menu_Identificação

```
var n1, opção: inteiro;
início
    escreva("Entre com o valor a ser testado: ");
    leia(n1);
    escreva("Opção 1: Par ou ímpar");
    escreva("Opção 2: Positivo, negativo ou 0");
    escreva("Opção 3: Bissexto");
    leia(opção);
    caso opção seja:
        1:
            se (n1 resto 2) = 0 então
                escreva("Número par.");
            senão
                escreva("Número ímpar.");
        2:
            se (n1 > 0) então
                escreva("Número positivo.");
            senão se (n1 < 0) então
                escreva("Número negativo.");
            senão
                escreva("Número 0.");
        3:
            se ((n1 resto 4 = 0 e n1 resto 100 <> 0)
                ou (n1 resto 400 = 0)) então
                escreva("Ano bissexto.");
            senão
                escreva("Ano não é bissexto.");
    default:
        escreva("Opção inválida.");
fim
```

■ Escopo da negação do bissexto — A condição $(n1 \text{ resto } 4 = 0 \text{ e } n1 \text{ resto } 100 \neq 0) \text{ ou } (n1 \text{ resto } 400 = 0)$ é cuidadosa com os parênteses: sem eles, a precedência de e ou mudaria o significado. Os parênteses são obrigatórios.

■ Precedência em C — Em C, $\&\&$ tem precedência maior que $\|$. Portanto $n1 \% 4 == 0 \&\& n1 \% 100 != 0 \| n1 \% 400 == 0$ seria avaliado como $(A \&\& B) \| C$ — o que é o que queremos. Ainda assim, os parênteses melhoram a legibilidade. Nunca confie na memória sobre precedência: parentize.

Mapa de equivalências: Portugal ↔ C

Portugol	C
se <cond> então ... senão	if (<cond>) { ... } else { ... }
senão se	else if
caso x seja: ... default:	switch (x) { case: default: }
início ... fim (bloco)	{ ... }
← (atribuição)	=
= (igualdade)	==
<> (diferente)	!=
e, ou, não	&&, , !

Portugol	C
resto	%
div	/ (em inteiros)
V, F	true, false (<stdbool.h>)
constante X = 18;	#define X 18 ou const int X = 18;
escreva(...) / leia(...)	printf(...) / scanf(...)

Boas práticas do capítulo

■ Cinco lições para escrever decisões — 1) Endentação é obrigatória — é ela que torna legível o bloco subordinado a cada se. 2) Só use início/fim quando o bloco tiver mais de uma instrução. 3) Prefira encadeamento (senão se) a ifs independentes quando sabe que apenas uma condição será verdadeira. 4) Use comentários de invariantes ({ n1 <> n2 }) para documentar o que é logicamente garantido. 5) Use caso apenas para comparação de igualdade contra constantes; para faixas ou expressões, use se encadeado.

Armadilhas ao traduzir para C

■ Checklist ao portar portugol → C — • Trocar = por == em comparações. • Usar && e || (curto-circuito), nunca & e | (bit-a-bit). • Incluir break em cada case do switch. • switch só aceita tipos integrais (int, char, enums) — não aceita float, double, strings. • Usar chaves { } ao redor de blocos com mais de uma instrução. • % em C é o resto da divisão inteira; para reais existe fmod() em <math.h>. • scanf("%d", &var) exige o endereço da variável (&); esquecer o & é erro silencioso.

Resumo Rápido

```

Comandos de Condição (Medina - Cap. 3)
■■■ 3.1 Comando se
■   ■■■ Sintaxe: se <cond> então [senão] ... fim
■   ■■■ Condição = expressão booleana (V ou F)
■   ■■■ senão é opcional
■   ■■■ Exemplos Verifica5
■   ■   ■■■ Versão 1: sem senão
■   ■   ■■■ Versão 2: com senão (blocos mutuamente exclusivos)
■   ■■■ Problema 5: Maior_Idade (constante LIMITE = 18)
■   ■■■ Problema 6: Maior_Versão1 (dois números)
■   ■
■   ■■■ 3.1.3 Condições compostas
■   ■   ■■■ Operadores: e, ou, não
■   ■   ■■■ Problema 7 V1: ifs independentes (11 instr.)
■   ■   ■■■ Erro comum: (a = b = c) → usar (a=b e b=c)
■   ■   ■■■ Igualdade é transitiva → um termo redundante
■   ■
■   ■■■ 3.1.4 Comandos encadeados
■   ■   ■■■ se dentro de se (aninhamento)
■   ■   ■■■ senão se (alinhado na mesma coluna)
■   ■   ■■■ Problema 7 V2: encadeada + compostas (9 instr.)
■   ■   ■■■ Problema 7 V3: encadeada sem compostas
■   ■   ■■■ senão não tem custo computacional
■   ■   ■■■ Problema 8: Maior_Versão2 (trata iguais)
■   ■
■■■ 3.2 Comando caso
■   ■■■ Especialização de se encadeado
■   ■■■ Compara variável inteira/caractere com constantes
■   ■■■ Sintaxe: caso x seja: v1: ... default: ...
■   ■■■ Restrição: só igualdade, só inteiro/caractere
■   ■■■ Problema 9: Exemplo_Caso (1, 5 ou 10)
■   ■■■ Problema 10: Menu_Identificação
■       ■■■ Par/ímpar (resto 2)
■       ■■■ Positivo/negativo/zero (encadeado)
■       ■■■ Ano bissexto:  $(\div 4 \wedge \neg \div 100) \vee \div 400$ 

```

#algoritmos #programação #medina #estruturas-de-controle #condicionais #se-senao #caso-switch #condições-compostas
#encadeamento #teste-de-mesa #portugol #linguagem-c